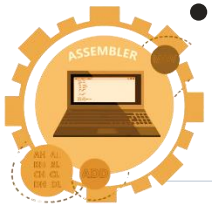
Lecture # 3

Organization of of the IBM Personal Computers

Engr. Fiaz Khan

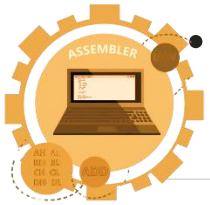
The Intel 8086 Family of Microprocessors

- The IBM PC family includes several models: IBM PC, PC XT, PC AT, PS/1, and PS/2.
- These computers are all built around a group of Intel 8086 microprocessors.
- Here's how they match up:
- IBM PC and PC XT use the 8088 microprocessor.
- PC AT and PS/1 use the 80286 microprocessor.
- Some PC-compatible laptops use the 80186 microprocessor.
- PS/2 models can have either the 8086, 80286, 80386, or 80486.



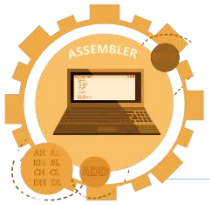
The 8086 and 8088 Microprocessors

- In 1978, Intel introduced the 8086, its first 16-bit microprocessor. A 16-bit processor can handle 16 bits of data at once.
- The 8088 followed in 1979. Internally, it's almost the same as the 8086.
- Externally, the key difference is that the 8086 has a 16-bit data bus, while the 8088 has an 8-bit data bus.
- Additionally, the 8086 runs at a faster clock rate, which means better performance.
- IBM's Choice:
- When creating the original IBM PC, they opted for the 8088 over the 8086.
- Why? Because it was cheaper to build a computer around the 8088.
- Common Instructions:
- Both the 8086 and 8088 share the same set of instructions.
- They form the foundation for instructions used by other microprocessors in the same family.



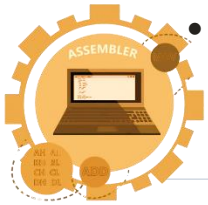
The 80186 and 80188 Microprocessors

- 80186 and 80188:
- These are enhanced versions of the 8086 and 8088 microprocessors, respectively.
- Their advantage lies in incorporating all the functions of the 8086 and 8088, along with some additional support chips.
- They can also execute a set of new instructions called the extended instruction set.
- Comparison with 8086 and 8088:
- Unfortunately, these processors didn’t offer any significant advantage over the 8086 and 8088.
- Eventually, they were overshadowed by the development of the 80286 microprocessor.



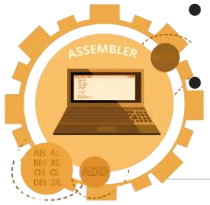
The 80286 Microprocessor

- Introduction and Speed:
- The 80286, introduced in 1982, is a 16-bit microprocessor.
- Unlike its predecessor, the 8086, it can operate at a faster clock rate (12.5 MHz vs. 10 MHz).
- Operating Modes:
- The 80286 has two modes of operation:
- Real address mode: In this mode, it behaves like the 8086. Programs written for the 8086 can run without changes.
- Protected virtual address mode (protected mode): Here, the 80286 supports multitasking (running several programs simultaneously) and memory protection (safeguarding memory from other programs).
- Memory Capabilities:
- In protected mode, the 80286 can address up to 16 megabytes of physical memory (compared to 1 megabyte for the 8086 and 8088).
- It can also treat external storage (like a disk) as if it were physical memory, allowing execution of programs up to 1 gigabyte in size.



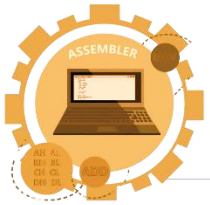
The 80386 and 803B6SX Microprocessors

- Introduction and Speed:
- In 1985, Intel introduced its first 32-bit microprocessor, the 80386 (also known as 386).
- It's much faster than the 80286 due to several reasons:
- 32-bit data path: It can handle more data at once.
- High clock rate: It operates at speeds up to 33 MHz.
- It can execute instructions more efficiently than the 8086.
- Operating Modes:
- Like the 80286, the 386 can operate in real mode (similar to the 8086) or protected mode (emulating the 80286).
- It also has a special virtual 8086 mode to run multiple 8080 applications under memory protection.
- Memory Capabilities:
- In protected mode, the 386 can address 4 gigabytes of physical memory and 64 terabytes of virtual memory.
- The 386SX is similar internally but has only a 16-bit data bus.



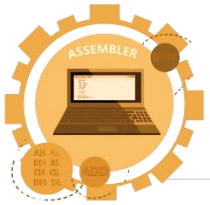
The 80486 and 80486SX Microprocessor

- Introduction and Speed:
- Introduced in 1989, the 80486 (486) is another 32-bit microprocessor.
- It’s the fastest and most powerful processor in its family.
- Key Features:
- The 486 combines the functions of the 80386 with additional support chips.
- It includes the 80387 numeric processor, which handles floating-point number operations.
- The 486 also has an 8-KB cache memory that acts as a fast buffer for data from slower memory.
- Performance:
- Thanks to its numeric processor and cache memory, the 486 is three times faster than a 386 running at the same clock speed.
- There’s also a variant called the 486SX, which lacks the floating-point processor.



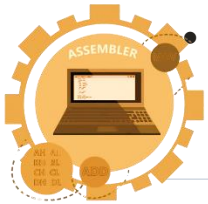
Organization of the 8086/8088 Microprocessors

- Focus on 8086 and 8088:
- The rest of this chapter will delve into the organization of the 8086 and 8088 microprocessors.
- These processors have the simplest structure.
- Most of the instructions we'll study are 8086/8088 instructions.
- Insight for Advanced Processors:
- Understanding the 8086 and 8088 provides insight into the organization of more advanced processors, discussed in Chapter 20.
- Name Clarification:
- Since the 8086 and 8088 share essentially the same internal structure, we'll use the name "8086" to refer to both.



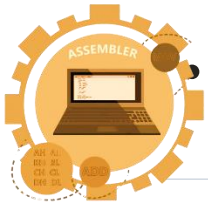
Registers

- Inside a microprocessor, information is stored in registers.
- Registers serve different functions based on their purpose.
- Data Registers:
- Data registers hold data for various operations.
- Address Registers:
- Address registers store memory addresses for instructions or data.
- Program Counter (PC):
- The PC register keeps track of the current state of the processor.
- 8086 Registers:
- The 8086 microprocessor has four general data registers.
- Address registers include segment, pointer, and index registers.
- The FLAGS register contains status information.

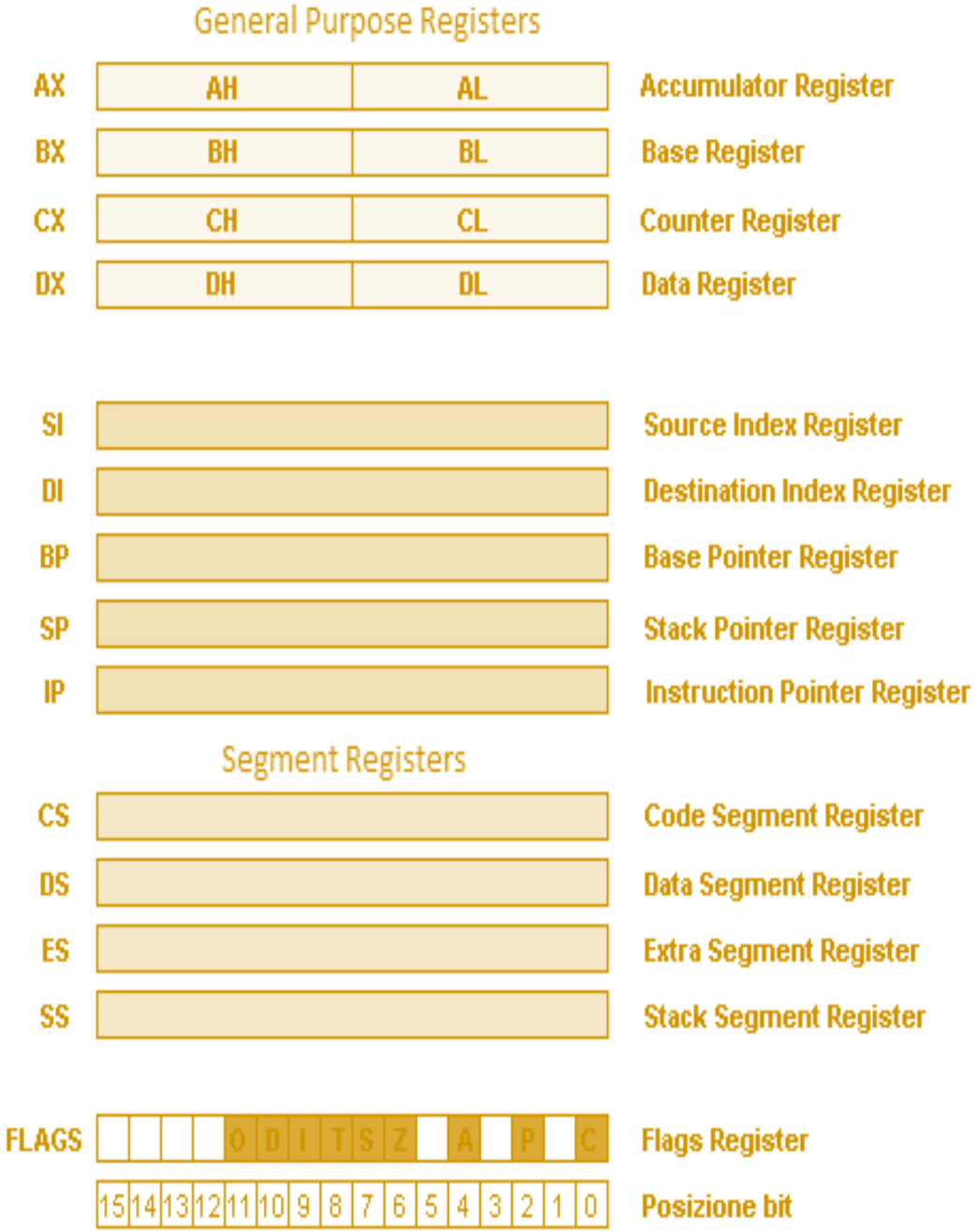


Data Registers: AX, BX, CX, DX

- General Purpose Registers
- Programmers can use these four registers for data manipulation.
- Storing data in registers is faster than accessing data from memory.
- Modern processors have many registers for efficiency.
- High and Low Bytes:
- Each data register has a high byte (called H) and a low byte (called L).
- For example, in AX, AH represents the high byte, and AL represents the low byte.
- This arrangement provides more flexibility when dealing with byte-size data.
- Special Functions:
- Besides their general-purpose role, these registers also serve special functions.
- We'll explore these functions as we go along.

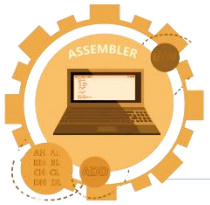


Data Registers: AX, BX, CX, DX



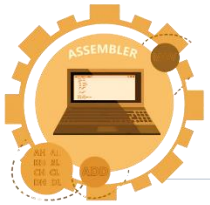
AX (Accumulator Register):

- AX is a versatile register used for arithmetic, logic, and data transfer instructions.
- It’s especially handy for multiplication and division operations.
- When working with numbers, one of them should be in AX or AL (its low byte).
- Input/output operations also involve AL and AX.



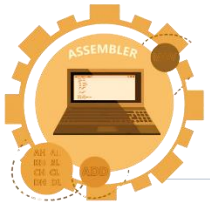
BX (Base Register):

- BX serves as an address register.
- For example, it’s used in an instruction called XLAT (translate).



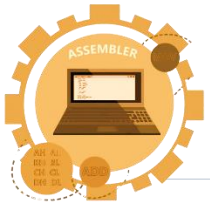
CX (Count Register):

- CX acts as a loop counter for program loops.
- It’s also used in repeat instructions that handle a special class of operations called string operations.
- CL (its low byte) counts in instructions that shift and rotate bits.



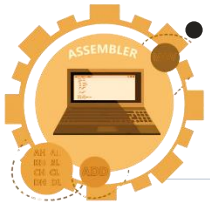
DX (Data Register):

- DX comes into play during multiplication and division.
- It’s also involved in input/output operations.



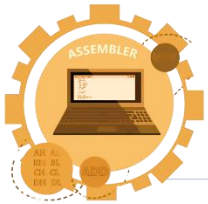
Segment Registers: CS, DX, SS, ES

- **Address Registers and Memory:**
- Address registers store the memory addresses of instructions and data.
- These addresses help the processor access specific memory locations.
- **Memory Basics:**
- Memory is like a collection of bytes (tiny storage units).
- Each byte has an address, starting from 0.
- The 8086 processor uses a 20-bit physical address for memory locations.
- This allows addressing up to 1,048,576 bytes (one megabyte) of memory.



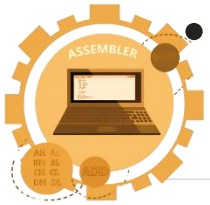
Segment Registers: CS, DX, SS, ES (Conti)

- **Hexadecimal Addresses:**
- Binary addresses are cumbersome to write, so we often use hexadecimal (base-16) notation.
- For example, the first five bytes have addresses like 00000, 00001, 00002, and so on.
- The highest address is FFFFh (meaning all 1s in hexadecimal).
- **Memory Segments:**
- The 8086 faces a challenge: its 20-bit addresses don't fit neatly into 16-bit registers.
- So, it partitions memory into segments.
- Each segment is like a separate chunk of memory, and the processor works with these segments.



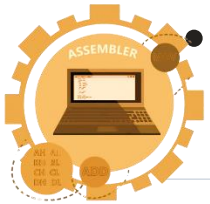
Memory Segment

- A memory segment is a block of 64 KB (kilobytes) or 2^16 consecutive memory bytes.
- Each segment is identified by a segment number, starting from 0.
- The segment number is 16 bits, so the highest segment number is FFFFh.
- **Addressing Within a Segment:**
- Within a segment, a memory location is specified by an offset.
- The offset represents the number of bytes from the beginning of the segment.
- For example, the first byte in a segment has an offset of 0.
- The last offset in a segment is FFFFh (meaning all 1s in hexadecimal).



Segment: Offset Address

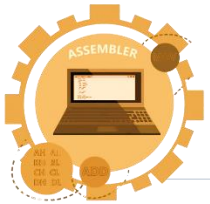
- **Logical Address Format:**
- A memory location is specified using a segment number and an offset.
- This format looks like: Segment:Offset.
- For example, A4FB:4872h means offset 4872h within segment A4FBh.
- Obtaining the Physical Address:
- The 8086 microprocessor needs to convert this logical address to a 20-bit physical address.
- First, it shifts the segment address 4 bits to the left (like multiplying by 16).
- Then, it adds the offset to get the final physical address.



Segment: Offset Address - Example Calculation

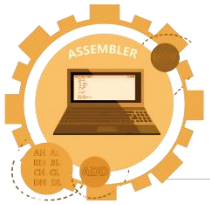
- For A4FB:4872:
- Shifted segment address: A4FB0h
- Added offset: +4872h
- Resulting physical address: A9822h (a 20-bit address)

- Note: Logical addresses help us find memory locations, and converting them to physical addresses involves shifting and adding!



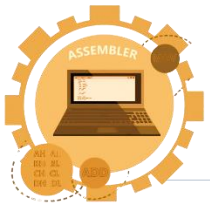
Location of Segments

- Memory Segments:
- Memory is divided into segments.
- Each segment starts at a specific address and contains consecutive memory bytes.
- For example, Segment 0 starts at address 0000:0000 (hexadecimal) and ends at FFFF:OFFF.



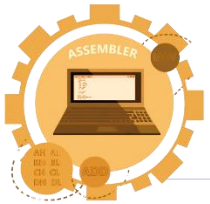
Location of Segments - Overlapping Segments

- There’s overlap between segments.
- Segment 1 starts at address 0001:0000 and ends at 0001:FFFF.
- Notice that segments start every 16 bytes (which we call a paragraph).
- An address divisible by 16 (ending with a hex digit 0) is called a paragraph boundary.
- Unique Addresses:
- The segment:offset form of an address is not always unique.
- Different combinations of segment and offset can point to the same physical memory location.



Example:
For the memory location whose physical address is specified by 1256Ah, give the address in segment:offset form for segments 12560h and 1240h.

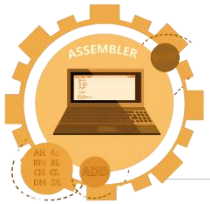
- Let’s break down the solution step by step:
- Given Information:
- The physical address is specified as 1256Ah.
- Segment:Offset Form:
- We want to express this address in segment:offset form for segments 12560h and 1240h.



- Calculating Offsets:
- Let’s find the offsets for each segment:
- For segment 1256h:
- Offset X = 1256Ah - 12560h = 000Ah (hexadecimal)
- For segment 1240h:
- Offset Y = 1256Ah - 12400h = 016Ah (hexadecimal)
- Final Addresses:
- The address in segment:offset form:
- For segment 1256h: 1256:000A
- For segment 1240h: 1240:016A

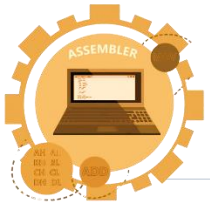
Program Structure:

- A typical machine language program consists of instructions (code) and data.
- There's also a stack used by the processor for procedure calls.



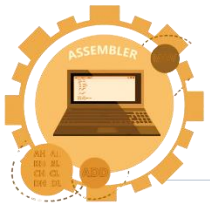
Memory Segments:

- The program's code, data, and stack are loaded into different memory segments:
- Code segment: Holds the executable program code.
- Data segment: Stores data.
- Stack segment: Used for stack data.
- Extra segment (ES): Can be used for additional data segments.



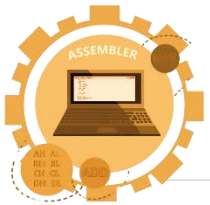
Overlapping Segments:

- Program segments don’t need to occupy the entire 64 kilobytes in a memory segment.
- Overlapping segments allow smaller segments to be placed close together.



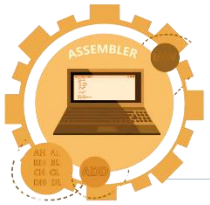
Segment Registers:

- The 8086 has four segment registers:
- CS: Holds the code segment number.
- DS: Holds the data segment number.
- SS: Holds the stack segment number.
- ES: Can be used for an extra data segment.



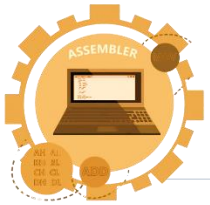
Active Segments:

- At any time, only the memory locations addressed by these four segment registers are accessible.
- But a program can modify the contents of a segment register to address different segments.
- Note: memory segments help organize program data, and segment registers keep track of these segments!



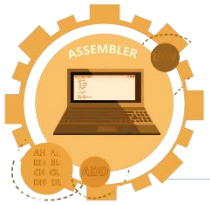
Pointer and Index Registers: SP, BP, SI, DI

- At any time, only the memory locations addressed by these four segment registers are accessible.
- But a program can modify the contents of a segment register to address different segments.
- Note: memory segments help organize program data, and segment registers keep track of these segments!



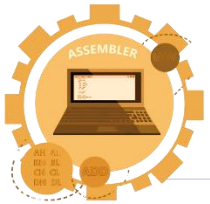
SP (Stack Pointer):

- The SP register points to the topmost item in the stack.
- It’s used for stack operations (like pushing and popping data).
- If the stack is empty, the stack pointer is FFFEh (hexadecimal).



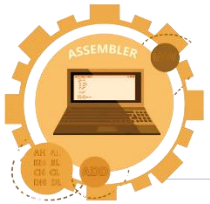
BP (Base Pointer):

- BP is primarily used to access data on the stack.
- Unlike SP, we can also use BP to access data in other segments.



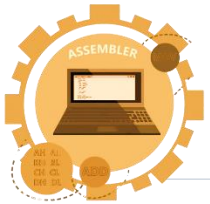
SI (Source Index):

- SI points to memory locations in the data segment addressed by DS.
- It helps access consecutive memory locations easily.



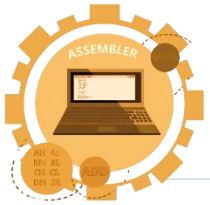
DI (Destination Index):

- DI performs the same functions as SI.
- It’s used for pointer addressing of data and string-related operations.



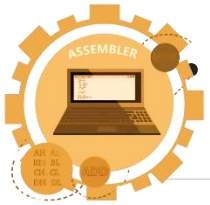
Instruction Pointer(IP) & CS

- CS (Code Segment):
- CS holds the segment number of the next instruction.
- It helps locate the code segment where the next instruction resides.
- IP (Instruction Pointer):
- IP contains the offset within the code segment.
- It’s updated after each instruction execution to point to the next instruction.
- Unlike other registers, IP cannot be directly manipulated by an instruction.



FLAGS Register

- Purpose of FLAGS Register:
- The FLAGS register reflects the status of the microprocessor.
- It does this by setting individual bits called flags.
- Status Flags:
- These flags represent the result of an instruction executed by the processor.
- For example, when a subtraction operation results in 0, the ZF (zero flag) is set to 1 (true).
- A subsequent instruction can check the ZF and take action based on a zero result.
- Control Flags:
- Control flags enable or disable specific operations of the processor.
- For instance, if the IF (interrupt flag) is cleared (set to 0), keyboard inputs are ignored by the processor.
- Note: FLAGS register helps manage the processor's status and control various operations!

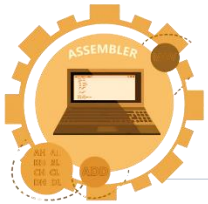


Organization of the PC

- Computer System Components:
- A computer system consists of both hardware and software.
- Software controls hardware operations, making it essential to understand both aspects.

- Operating system

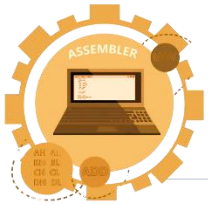
- Operating System (OS):
- The most crucial software for a computer is the operating system (OS).
- The OS coordinates all devices in the computer system.
- OS functions include reading/executing user commands, I/O operations, error messages, and memory/resource management.



Organization of the PC -Conti

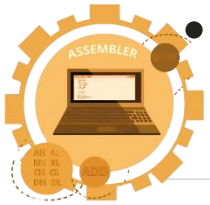
- Disk operating system

- DOS (Disk Operating System):
- DOS is a popular OS for IBM PCs (8086/8088-based computers).
- It manages up to 1 megabyte of memory and doesn’t support multitasking.
- DOS reads/writes information on disks, organizes data into files, and has various versions.
- The latest version, DOS 5.0, even supports a graphical user interface (GUI) with mouse input.



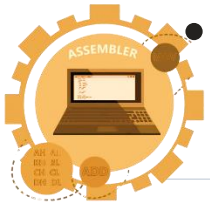
Organization of the PC - Memory Map and Segments:

- The 8086/8088 processor can address 1 megabyte of memory.
- Not all memory is usable by application programs.
- Some locations have special meanings (e.g., interrupt vectors).
- Display memory holds data shown on the monitor.
- Disjoint Memory Segments:
- To show the memory map, we partition it into disjoint segments.
- Segment 0 ends at OFFFFh, and the next segment starts at 1000:0000.
- There are 16 disjoint segments in total.



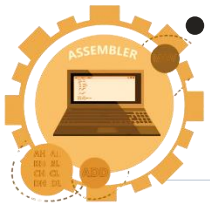
Organization of the PC - DOS

- DOS and Memory Sizes:
- DOS uses the first 10 disjoint memory segments for loading and running application programs.
- These segments (from OOOOh to 9000h) provide 640 KB of memory.
- Other segments are reserved or used for video display memory.
- Segment FOOOh contains BIOS routines and ROM BASIC.
- Common I/O Ports:
- Various I/O ports (e.g., interrupt controller, keyboard controller) serve specific purposes.
- These ports allow communication with external devices.



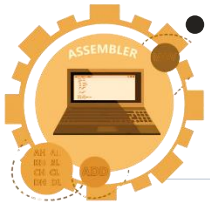
Organization of the PC - I/O Ports and Port Addresses:

- The 8086/8088 supports 64 KB of I/O ports.
- Common port addresses are given in Table 3.1.
- Direct programming of I/O ports is not recommended due to variations among computer models.
- PC Boot Process and BIOS:
- When the PC powers up, the 8086/8088 processor starts in a reset state.
- The CS register is set to FFFFh, and IP is set to 0000h.



Organization of the PC

- The first instruction executed is in ROM, transferring control to the BIOS routines.
- BIOS checks for errors, initializes interrupt vectors, and loads the operating system from the system disk.
- Boot Program and Operating System:
- The boot program (part of the OS) loads the actual OS routines.
- BIOS isolates itself from OS changes.
- After loading, COMMAND.COM takes control.



ENGR. MUHAMMAD FIAZ

LECTURER COMPUTER SCIENCE

TELF/EF SET / ACEPT CERTIFIED

MICROSOFT ACADEMY TRAINER

ORACLE CERTIFIED PROFESSIONAL

GOOGLE CERTIFIED PROFESSIONAL

Lahore, Pakistan

+92-320-7617-093

muhammad.fiaz@cs.uol.edu.pk

https://www.linkedin/in/fiazofficials

THANK YOU!

Do you have any questions?